1 Research Question and Task

Selecting Reliable LLM Reasoning with Candidate Attention and Reliability Features

DASE7506 MP2 | Advanced Machine Learning Methods
Author: wuhaonan
Student ID: 3036843288
Version: mp2-attention-reliability-v1-20261021
Status: validation and final test evaluated; model frozen before test scoring.

Abstract

This project selects one of four existing solutions to a mathematics question. It investigates whether comparisons between candidates and explicit reliability signals improve selection accuracy. The final supervised ranker combines the supplied frozen embeddings with text statistics, answer agreement, parsing-format indicators and local arithmetic checks. A compact single-head attention network achieves 743/798 (93.11%) on validation, compared with 696/798 (87.22%) for the initial logistic-regression baseline. Its mean validation accuracy across five training seeds is 92.36%. Fixed equal-weight ensembles do not outperform the primary-seed single model. The frozen primary model subsequently achieves 381/467 (81.58%) on the complete test bank. Test results were not used for model selection or tuning.

Data and supervision

The supplied bank contains 4,755 training, 798 validation and 467 test questions, each with four candidate solutions and a 1,024-dimensional embedding per candidate. Training labels are obtained by comparing each candidate's numeric answer, extracted by the unchanged official parser, with the training reference. Multiple candidates can be positive. Labels supervise final-answer correctness; they do not establish that every reasoning step is valid.

Only supplied training inputs and references are used to fit parameters and normalization statistics. Validation is used for development, model selection and error analysis. Test inputs and references were withheld during development and used only for final prediction and independent scoring on 26 October 2026. There are no external training examples, additional pretrained weights, online LLM calls or answer lookup tables. IDs serve alignment only.

Hypothesis and scope

The working hypothesis is that candidate-to-candidate comparison and computable reliability signals help distinguish convincing but incorrect solutions. Experiments compare logistic regression, MLP losses, attention, tree models and ensembles. Validation has been consulted across several development rounds, so the selected result may be optimistic. Failed approaches and seed variability are therefore reported alongside the best score.

The investigation has three questions: (RQ1) is joint candidate scoring useful in the tested model family? (RQ2) do explicit format and arithmetic signals help beyond base features? (RQ3) does additional inference computation yield a net benefit? Existing comparisons support design choices, but do not isolate every mechanism causally.

2 Model and Supervised Objective

Representation and architecture

Each candidate has 1,056 input features: the supplied 1,024-dimensional embedding, 16 base features, eight format-risk features and eight arithmetic features. Per-feature means and standard deviations are fitted on training candidates only; standard deviations are floored at 1e-5 and reused at inference.

A shared linear layer maps each candidate from 1,056 to 64 dimensions, followed by ReLU and training-time dropout of 0.15. Bias-free projections produce Q, K and V across the four candidates. Attention uses softmax(QK^T / sqrt(64)). Each candidate's own hidden representation is concatenated with its attention context, producing 128 dimensions for a shared linear scoring head. No positional embeddings are used.

Loss for multiple correct candidates

Let s_j be the score of candidate j and C the nonempty set of correct candidates. The loss for one question is:

L = log(sum_j exp(s_j)) - log(sum_{j in C} exp(s_j))

This maximizes the total softmax probability assigned to all correct candidates rather than forcing a single positive target. If all four candidates are correct, this loss is zero. It is not the same as applying softmax directly to binary relevance labels in ListNet.

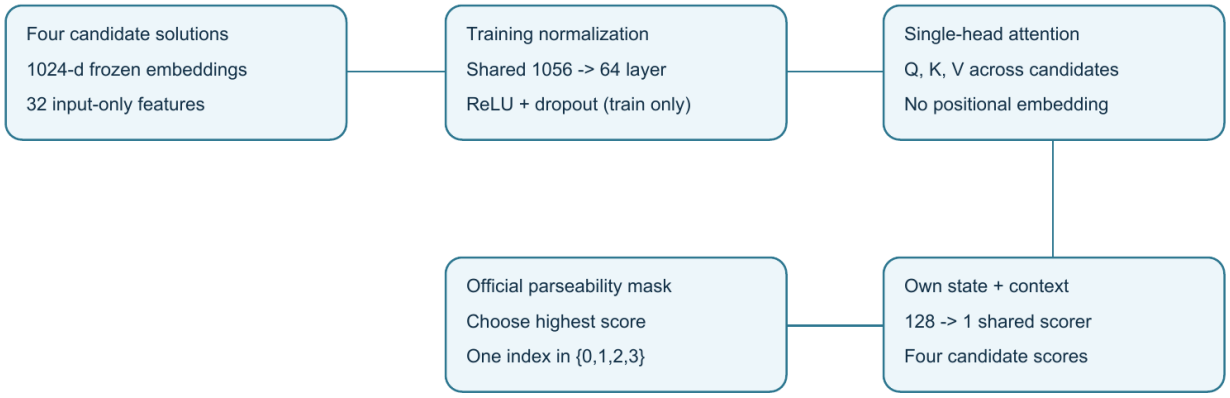
Setting	Value
Optimizer	AdamW; learning rate 0.001; weight decay 0.01
Batch / gradient clipping	128 questions / maximum norm 5
Hidden size / primary seed	64 / 7506
Full-training epochs	5
Compute	CPU; four threads

An initial question-level split of training data, using seed 7506, allocated 3,804 questions to fitting and 951 to internal validation. Candidates from one question stayed together. The original attention model selected five epochs internally and was refitted on all training questions. Later feature and seed comparisons retained this epoch count. The final standalone train.py loads training data only.

Inference disables dropout and evaluates the network in NumPy. Candidates that the official parser cannot parse are masked before selection; the training loss still includes all four scores. If all candidates in an unseen group are unparseable, selection falls back to model scores. The output is one Python integer index per question, with no answer generation.

3 How the Selector Makes a Decision

Frozen selector: candidate comparison + reliability signals



Training: maximize total probability of all correct candidates. Inference: no references.

RQ1: candidate comparison

The attention model combines each candidate with information from the other three. The original attention configuration achieves 92.36% validation accuracy versus 90.85% for the tested correct-set-loss MLP. Inputs and architectures differ, so this is suggestive evidence, not a controlled estimate of the effect of attention alone.

What is learned and what is fixed?

The projection, attention maps and scoring head are learned from training labels. Parsing and numeric checks are fixed, reference-free preprocessing rules. The final choice remains one of the supplied candidates. No arithmetic repair or new answer is generated.

4 Reliability Features and Limitations

Base features: 16 dimensions

These include parseability, the fraction of candidates sharing an answer, the number of distinct parsed answers, character/word/line/number counts, relative character and word counts, question-word overlap, conservative binary-equation counts, and mean/maximum/minimum cosine similarities between candidate embeddings. All are computed from the current inputs without references.

Format risk: eight dimensions

Indicators capture the presence and count of #### markers, whether the final marker resembles a numbered section heading, boxed-answer presence, disagreement between boxed answers, disagreement between the last boxed value and the official parsed value, whether a boxed answer follows the last #### marker, and official parse failure. These features do not change the parser or substitute a different answer for grading.

Arithmetic evidence: eight dimensions

The checker returns counts of checked, consistent and inconsistent numeric equalities; the inconsistent fraction; whether any inconsistency exists; whether the last checked equality is inconsistent; and mean/maximum relative error. Counts use log1p. Supported expressions include multi-term arithmetic, parentheses, bounded powers, LaTeX fractions and multiplication/division symbols, and adjacent numeric expressions within equality chains.

Expressions are evaluated using restricted AST traversal and Fraction arithmetic, without eval, exec or execution of candidate code. Explicit decimal precision permits corresponding rounding tolerance. Variables, unsupported LaTeX, unit-text boundaries, percentage-containing equalities and ambiguous leading list hyphens are conservatively skipped. Integer rounding can still trigger false alarms. The checker does not verify units or semantic relationships, and its output is a learned feature rather than a hard rejection rule.

Implementation revisions and coverage

Training-text inspection exposed false alarms involving Markdown bullets, unit boundaries and percentages in the first implementation. The second was too conservative around inline mathematics and equality-chain interiors. The third preserves mathematical boundaries and skips only affected adjacent pairs. All three versions were evaluated on validation, and both earlier versions are archived. This development history is disclosed rather than treating the final version as an untouched preregistered experiment.

The final checker examines at least one equality in 2,308 of 3,192 validation candidates and flags 66 candidates. Eight flagged candidates nevertheless have officially correct final answers, so trigger counts are not final-answer error precision. Among the original attention model's 61 selected wrong candidates, 44 have a checkable equality and four contain a detected numeric inconsistency.

5 Validation Comparisons and Ablations

All results below use the official validation scorer on 798 questions. A diagnostic subset contains 355 questions with at least two distinct parseable numeric answers; the other 443 are easier. This subset is not the test bank and cannot replace test evaluation.

Method (primary seed 7506)	Correct	Accuracy
Initial logistic regression	696	87.22%
Initial model + parseability filter	711	89.10%
L2 logistic regression	716	89.72%
MLP: binary cross-entropy	715	89.60%
MLP: pairwise ranking	716	89.72%
MLP: correct-set probability loss	725	90.85%
MLP with relative features	731	91.60%
Original attention + base features	737	92.36%
LightGBM binary / ranking	701 / 704	87.84% / 88.22%
Attention + format features only	730	91.48%
Attention + arithmetic features only	737	92.36%
Attention + both feature groups	743	93.11%

Interpretation

Parseability filtering alone adds 15 correct selections, so the full improvement should not be attributed to neural modeling. The correct-set probability objective outperforms the tested BCE and pairwise MLP variants. Attention outperforms the other tested configurations, but differences in architecture, dimensions and optimization mean this is not a fully controlled causal comparison.

Neither added feature group alone exceeds the original primary-seed attention result. Combining them improves it by six questions, or 0.75 percentage points. Relative to the initial baseline, the improvement is 47 questions, or 5.89 percentage points. The final primary model achieves 300/355 (84.51%) on the distinct-answer subset; easier questions raise overall validation accuracy to 93.11%.

Logistic regression compares C in {0.001, 0.01, 0.1, 1, 10}. Tree models compare seven and fifteen leaves with internal early stopping and at most 500 rounds. Initial neural searches allow up to 50 epochs with ten epochs of patience. Later ablations and seed repetitions fix five epochs. Configurations and loss histories are retained in records/.

Final held-out test: 381/467 (81.58%). All test questions have at least two distinct parseable answers, unlike the full validation bank. The closer diagnostic comparison is the primary model's 84.51% on the 355-question distinct-answer validation subset. Different questions and repeated validation-based selection also limit direct comparisons; no single cause of the gap is established.

6 Stability, Ensembles and Errors

Training seed	Original attention	Final feature combination
7506	92.36%	93.11%
17	91.73%	92.36%
42	91.98%	92.61%
2026	91.60%	91.98%
3407	91.73%	91.73%
Mean / sample SD	91.88% / 0.30 pp	92.36% / 0.54 pp

Within each model family, seeds change initialization, dropout and training order while retaining hyperparameters. Added input dimensions also change initialization correspondence, so weights are not paired parameter by parameter. The final combination improves mean accuracy by 0.48 percentage points: four seeds improve and one ties, although variability increases. Distinct-answer subset means increase from 81.75% to 82.82%. These are training-randomness variations on one shared validation set, not generalization confidence intervals.

Fixed equal-weight ensembles

Ensemble	Validation accuracy
Original + new primary models	92.73%
Original model, five seeds	92.36%
New model, five seeds	92.73%
Original + new, ten models	92.36%

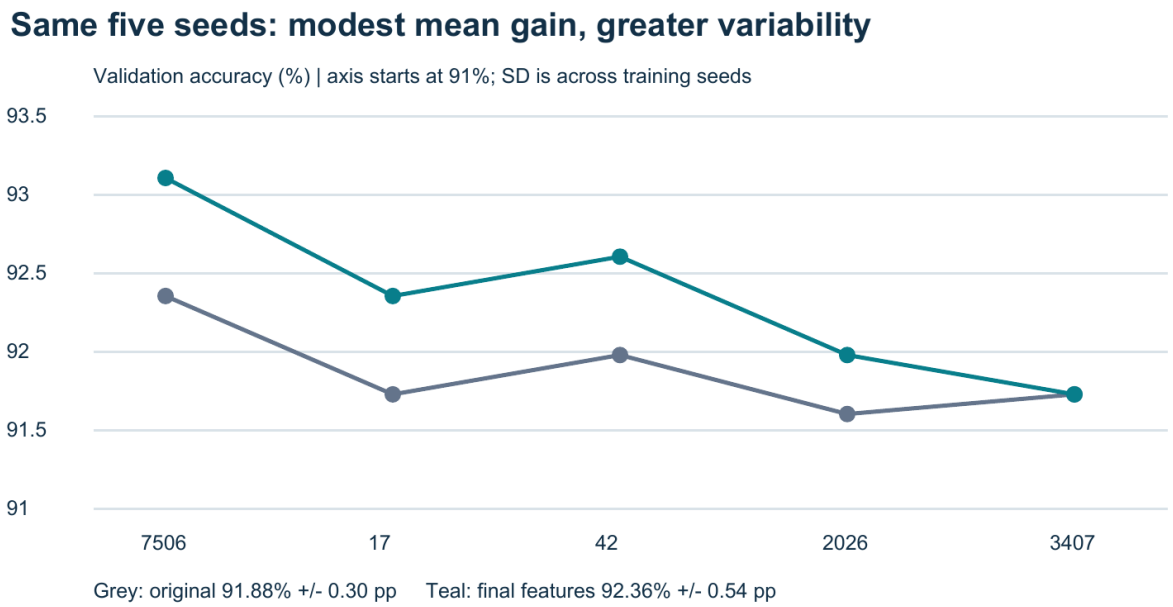
Each member masks unparseable candidates and applies softmax at temperature one; candidate probabilities are averaged equally. Four combinations were declared before scoring, with no weight search. The two-model ensemble fixes eight errors of the new single model but introduces eleven. The smaller, better-performing primary single model is retained.

Error analysis and scope

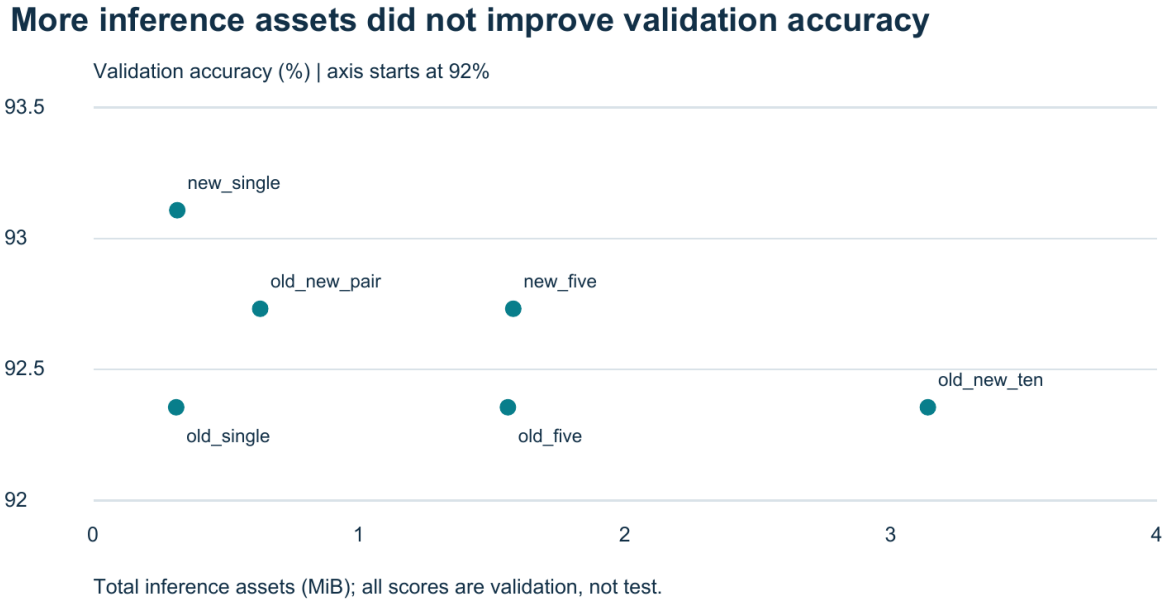
Reading the original attention model's 61 errors produced these primary categories: semantic relationships/counting (30), omitted conditions or final steps (16), numerical operations (10), units (3), suspected wording/reference conflict (1), and a mismatch between visible and officially parsed answers (1). These are Codex-assisted interpretations requiring student review, not a new categorization of the final model's 55 errors.

Examples include multiplying weekly hours by days, confusing pairs with people, and calculating tax without adding it. For validation index 733, the visible boxed answer is 250, but a #### Job 3 heading makes the official parser return 3. Indices 32 and 661 have suspected wording/reference inconsistencies; the chosen answer for 661 is wrong under either interpretation. Official labels remain unchanged. Of the original 61 errors, 42 persist across all five original-model seeds, illustrating the limits of seed changes for semantic failures.

7 Evidence for the Design Decisions



RQ2: adding both reliability feature groups improves four seeds and ties one, averaging +0.48 percentage points. Sample SD rises from 0.30 to 0.54 points. The primary seed fixes 18 old validation errors but introduces 12: six net improvements, rather than universal correction.



RQ3: the new single model achieves 93.11% validation accuracy with approximately 0.316 MiB. The ten-model ensemble uses 3.138 MiB and scores 92.36%. The two-model blend fixes eight new-single-model errors while introducing eleven. We retain the single model. No weights were tuned on test results.

8 Resources, Reproduction and Disclosure

Packaged-model evaluation

The independent submission package reproduces 743/798, with official accuracy 93.107769424%. The complete fresh prediction and independent scoring processes take 1.948 seconds locally. Total inference assets are 331,155 bytes (0.316 MiB), including weights, normalization statistics and configuration. Weights use uncompressed np.savez storage.

Hardware: AMD Ryzen 7 H 255 with Radeon 780M, Windows 11, CPU inference restricted to four threads. Inference uses NumPy 2.5.3. Training used Python 3.12.0 and PyTorch 2.14.1; historical comparisons also used scikit-learn 1.9.1 and LightGBM 4.7.0. Environment and timing records accompany the package. Recorded train_tune_seconds includes training/tuning and some export work, excludes dependency installation, and is distinct from separately measured evaluation time.

The course limits inference assets to 16 MiB and complete CPU test evaluation to 30 seconds on the reference Xeon Platinum 8457C with four threads in total. Final test accuracy is 81.58458244% (381/467). Fresh prediction took 2.827 seconds and independent scoring took 0.678 seconds, totaling 3.504 seconds locally. This is below 30 seconds locally; eligibility is determined on the reference server.

Reproduction and versioning

Frozen assets: model/config.json and model/weights.npz. SHA256 identities: MODEL_MANIFEST.json and FILES_SHA256.json. Repository: https://github.com/haonanwu869-rgb/DASE7506-MP2, tag v1.1 (git rev-parse v1.1^{commit}). Course data is obtained separately.

Install requirements-inference.txt under Python 3.12 and run code/evaluate.py --data-dir <course-data-path>. For training, install requirements-training.txt and run code/train.py --data-dir <course-data-path> --output retrained_model. README.md provides full commands. Inference never loads references; the separate scorer does. The standalone inference package was evaluated, while the newly extracted minimal training entry point has not been independently rerun. Historical training outputs are retained.

AI assistance, reuse and sources

Codex assisted with method implementation, experiment execution, error reading and report preparation. The author must understand and review the implementation and retain this substantive AI-assistance disclosure. No third-party algorithm repository code was directly copied. NumPy and PyTorch are software dependencies. Data, frozen embeddings and official parsing/scoring utilities come from the coursework package; GSM8K attribution and its MIT license are retained in attribution/. Assessment rules are taken from the supplied guide/GUIDE.md; detailed experimental evidence is in records/.

Remaining submission steps

The model was frozen before final test scoring; asset hashes are unchanged. README.md and PUBLICATION.md identify the code and checkpoint release. Review the materials and complete the separate course GitHub-issue submission.